

CAHIER DES CHARGES FONCTIONNEL ET TECHNIQUE

Projet: "PointeResto" - Plateforme de Découverte et de Référencement Gastronomique Local

Cahier des Charges: Projet Étudiant / Cadre Académique

Version: 6.0 (Spécifications Full-Stack et Mobile, PostgreSQL & Géolocalisation des Établissements)

1. Présentation générale et Objectif du Projet

L'objectif de ce projet est de concevoir et développer une application web Full-Stack et mobile, performante et accessible, baptisée "**PointeResto**". Cette plateforme est dédiée aux habitants et visiteurs de Pointe-Noire, leur permettant de facilement trouver, évaluer et localiser les restaurants, maquis et fast-foods de la ville.

La plateforme remplit deux rôles majeurs :

- **Côté Citoyens (Public):** Permettre aux utilisateurs de rechercher un restaurant par quartier, par spécialité (grillades, fruits de mer, fast-food), de consulter les menus, de laisser des avis (nominatifs ou anonymes) et de s'orienter via géolocalisation vers des établissements locaux.
- **Côté Restaurateurs et Modérateurs (Interne):** Fournir aux gérants d'établissements un outil collaboratif pour mettre à jour leurs horaires, publier leurs cartes/menus, regrouper les avis clients et suivre l'état d'avancement des éventuelles réservations.

L'accent est mis sur la simplicité de la navigation mobile, l'intégrité technique des avis publiés (lutte contre les faux avis) et la classification automatique pour identifier les lieux les plus populaires agissant localement.

2. Acteurs du Projet (MOA & MOE)

Pour ancrer ce projet dans un cadre professionnel et réaliste, la gouvernance est définie comme suit :

- **Maîtrise d'Ouvrage (MOA):** Mr Webster L'architecte, seigneur du Temple de Cartenon. Il agit en tant que client final (représentant le syndicat des hôteliers/restaurateurs ou la promotion touristique), exprime les besoins fonctionnels, valide les étapes clés, s'assure de l'ergonomie de la plateforme et facilite le double éveil du codeur.
- **Maîtrise d'œuvre (MOE):** Les étudiants. La MOE est responsable de la conception technique, de la modélisation de la base de données relationnelle, du développement du code (Frontend, Backend et Mobile), de la sécurité contre les injections malveillantes sur

les formulaires d'avis et du déploiement de la solution.

3. Exigences de la MOA (Sécurité & Contraintes Techniques)

La MOA impose le respect strict des critères de performance, d'intégrité et de réalisme suivants :

- **Authenticité et Réalisme des Données (Mock Data):** Les données injectées initialement (Seed) doivent simuler de vrais établissements et comportements couramment observés à Pointe-Noire (ex: "La Paillote sur Bd Dr Jacques Bouiti", "Le Derrick à Av. Massafi", "Le Comptoir", "Caicos Restaurant", ou un "Fast-food populaire sur le rond-point Lumumba"). Le tableau d'objets doit contenir au moins 15 à 20 entrées réalistes d'établissements et d'avis.
- **Architecture Technologique (Contraintes strictes):**
 - **Frontend:** Ultra-léger, adapté aux connexions mobiles 3G/4G des usagers en déplacement. **Aucun framework lourd (comme React ou Angular) n'est autorisé** pour cette phase afin de garantir la maîtrise absolue des langages de base (HTML5, CSS3/Tailwind, JavaScript Vanilla).
 - **Backend:** Développement d'une API REST robuste (ex: Node.js, PHP OO, Python) capable de gérer l'upload sécurisé des photos de plats et de filtrer les entrées pour éviter le dépôt de fichiers malveillants.
 - **Base de données:** Utilisation obligatoire de **PostgreSQL** pour stocker les relations complexes entre les utilisateurs, les restaurants, les avis, les menus et les quartiers.
- **Code Couleur et Disponibilité Visuelle:** L'état de l'établissement ou de son affluence doit être identifiable immédiatement :
 - **Statut d'ouverture:** Ouvert actuellement (Vert) | Ferme bientôt (Orange) | Fermé (Rouge).
 - **Gamme de prix / Type:** Économique/Fast-Food (Gris) | Restaurant Classique (Bleu) | Haute Gastronomie (Or).

4. Spécifications du Backend, Base de Données et Sécurité

4.1. Modélisation de la Base de Données (PostgreSQL)

Le schéma relationnel doit permettre de regrouper les établissements par zone géographique via des Workspaces (Quartiers) et de croiser l'analyse pour déceler les tendances :

- **Workspaces / Zones Géographiques:** Le découpage de la ville (ex: "Centre-Ville / Lumumba", "Tié-Tié", "Mpaka", "Côte Sauvage"). Un Workspace isole les données de restauration selon la zone.

- **Établissements (Dossiers):** Informations fournies par le restaurateur (Nom, adresse exacte, téléphone, description, coordonnées GPS).
- **Médias & Menus (Artefacts Numériques):** Photos des plats, PDF des menus, intérieurs des restaurants.
- **Modérateurs (Utilisateurs internes):** L'équipe gérant l'approbation des nouveaux restaurants inscrits sur la plateforme.

4.2. Sécurité de l'API REST

- L'accès aux formulaires de dépôt d'avis public doit être protégé contre le spam (ex: limitation de requêtes par IP).
- Les fichiers téléchargés (photos des plats) doivent subir une vérification stricte de l'extension et du type MIME (uniquement JPEG, PNG) côté Backend pour empêcher l'exécution de scripts malveillants (.php, .js) déguisés en images.
- Les routes de gestion administrative (/api/workspaces/:id/restaurants/manage) doivent être strictement réservées aux restaurateurs et modérateurs authentifiés via un jeton de session sécurisé.

5. Spécifications Fonctionnelles Détaillées (Interface Frontend)

Module Fonctionnel	Description des Fonctionnalités et Contraintes
1. Portail Public & Recherche Gastronomique	Une interface publique simplifiée pensée d'abord pour les smartphones (Mobile-First). L'utilisateur tape sa recherche, sélectionne son quartier, consulte les fiches restaurants (ex: menu, localisation) et peut glisser-déposer ses propres photos s'il souhaite laisser un avis.
2. Dashboard Interne du Restaurateur	Une fois connecté à son espace de gestion, le propriétaire accède à un tableau de bord listant ses statistiques sous forme de cartes dynamiques. Cliquer sur une option lui permet d'actualiser son menu, de consulter ses avis, d'y répondre et d'assigner un statut

	(ex: Ouvert/Fermé exceptionnellement).
3. Système de Recherche Avancé & Filtres	Un moteur de recherche dynamique. Contrainte essentielle : si un client recherche un "hamburger" ou un "poisson braisé", le système doit scanner l'ensemble des menus et tags du Workspace pour faire ressortir tous les établissements pertinents, classés par distance et note moyenne.
4. Liste des Lieux Favoris (localStorage)	L'utilisateur peut "marquer d'un drapeau" (Mettre en favoris) les restaurants qu'il souhaite tester ce week-end. Ces fiches sont mémorisées dans le localStorage de son navigateur mobile pour être ancrées au sommet de sa page d'accueil, facilitant l'accès à l'information (horaires, itinéraire) à chaque reconnexion hors-ligne.

6. Calendrier et Date de Livraison

Le planning impératif du projet est structuré ainsi, prenant en compte le démarrage immédiat :

- **Lancement officiel du projet:** Samedi 06 Juin 2026
- **Revue intermédiaire:** Lundi 08 Juin 2026 via le premier dépôt GitHub (Structure de la base PostgreSQL et squelette de l'API REST des établissements)
- **Date Limite de Livraison:** Mardi 16 Juin 2026 à 13h59.

7. Livrables Attendus sur le Compte GitHub

La MOE doit structurer son dépôt de manière professionnelle :

1. **Architecture des fichiers source:** Séparation nette entre /frontend (Vanilla HTML/CSS/JS pour l'interface de recherche) et /backend (API REST, gestionnaire d'upload d'images, scripts de migration de la base PostgreSQL).
2. **Un fichier README.md professionnel:** Rédigé en Markdown, détaillant l'architecture du système "PointeResto", le dictionnaire des données PostgreSQL, et les mesures de sécurité implémentées contre l'upload de faux menus.
3. **Historique des commits:** Un minimum de 10 commits réguliers, bien nommés, témoignant du développement incrémental de l'application.
4. **Déploiement en ligne:** Le portail public Frontend doit être accessible en ligne (ex: GitHub Pages) et connecté au serveur backend ainsi qu'à la base de données PostgreSQL hébergés sur une infrastructure Cloud (ex: Render, Railway) pour des tests complets sur

mobile dans les rues de Pointe-Noire.

5. **Modélisation de données:** Schéma de la structure des tables PostgreSQL (MCD/MPD pour les restaurants, menus et avis) inclus dans le dépôt sous format PNG ou JPEG et fichier startUML.
6. **Lien vers une vidéo de démonstration YouTube:** Vidéo présentant le parcours utilisateur complet : recherche d'un restaurant par un habitant depuis son smartphone, consultation du menu, puis connexion du restaurateur sur son tableau de bord interne pour mettre à jour ses horaires et répondre à un avis.